

Suspicion-Aware AI Agent for Social Deduction Games

A Research Project Plan

MLR 555 — Advanced Artificial Intelligence

Topic	AI Werewolf with Suspicion-Based Reasoning
Track	Reproduce and Extend a Paper
Reference Platform	AIWolf Project (Japan)
Implementation	Self-Contained Python Engine
Game Format	8-Player Werewolf (2 Wolves)
Base Agents	Random, Heuristic, Bayesian, MCTS, Logic-Based
Team Size	3 Members
Survey Paper Due	Week 11
Research Project Due	Week 14

1. Project Overview

This project investigates whether modeling deceptive intent as an explicit, modular reasoning component improves AI performance in social deduction games. Most existing AIWolf agents focus on Bayesian role inference: estimating *what role* each player likely has based on their actions. Our contribution is a complementary **suspicion module** that models *whether a player is behaving deceptively* regardless of their assigned role — capturing strategic manipulation that pure role inference misses.

Critically, suspicion is implemented as a **modular add-on** rather than a standalone agent. Any compatible base agent can optionally use the suspicion module. This allows direct paired comparisons between any agent and its suspicion-enhanced variant, isolating the contribution of the suspicion layer from other implementation details.

Research Questions

- **RQ1.** Does adding a suspicion module improve win rate across diverse base agent types?
- **RQ2.** Which behavioral deception patterns contribute most to suspicion accuracy?
- **RQ3.** Does the suspicion module specifically catch deceptive wolf strategies (bus-driving, false-claiming) that base agents miss?
- **RQ4.** What is the optimal balance between role inference and suspicion in the combined decision policy?

Paradigm Coverage

Our five base agents span the major paradigms of classical AI, allowing us to test whether suspicion-aware reasoning generalizes across fundamentally different approaches:

Agent	AI Paradigm
Random	No reasoning (floor baseline)
Heuristic	Pattern matching / rule-based
Bayesian	Probabilistic inference
MCTS	Simulation-based search
Logic-Based	Symbolic reasoning

2. Game Setup

We use an 8-player Werewolf configuration as the primary experimental setup. This format strikes the right balance between computational feasibility and experimental depth.

Why 8 Players?

Multi-wolf coordination becomes possible. With 2 werewolves, wolves can coordinate, defend each other, or strategically vote against teammates ("bus-driving"). This makes our Protection Patterns detector meaningful — in 5-player single-wolf games, this detector cannot trigger.

More rounds = more behavioral data. 8-player games typically last 4-5 days, compared to 2-3 days in 5-player games. This gives our detectors substantially more observations to identify deception patterns.

Computationally feasible. Each game takes approximately 10 seconds. 10,000 games complete in a single overnight run.

Role Distribution

Role	Count	Team	Special Ability
Villager	4	Village	None — vote during day
Werewolf	2	Wolf	Coordinate kills at night
Seer	1	Village	Divine one player per night
Possessed	1	Wolf	Human-aligned to Seer

Game Flow

Day 0: Roles assigned, agents initialize beliefs and suspicion scores.

Day Phase: 5 rounds of discussion (each agent talks once per round), then plurality voting eliminates one player.

Night Phase: Seer divines one target. Werewolves coordinate to kill one player.

Win Conditions: Village wins when all wolves are eliminated. Wolves win when wolves equal or outnumber non-wolves.

3. Base Agents

All agents implement the same interface (mirroring AIWolf), but differ in their internal reasoning. Three of the five agents (Heuristic, Bayesian, MCTS) can optionally use the suspicion module; Random has no decisions to enhance, and Logic-Based deals in certainties rather than degrees of suspicion.

Random Agent (Floor Baseline)

Votes uniformly at random for any alive player. No belief tracking, no reasoning. Provides a lower bound on performance — any useful agent must beat this. Standard practice in AI research papers to include a random baseline.

Heuristic Agent (Simple Baseline)

Tracks accusation counts per player and votes for the most-accused alive player. Demonstrates simple pattern-matching behavior. Tends to bandwagon onto whoever the group is already suspicious of.

$$\text{vote}(p^*) = \text{argmax over alive } p \text{ of } \text{accusation_count}(p)$$

Bayesian Agent (Strong Baseline)

Maintains a probability distribution over role assignments for every player. Updates beliefs using Bayes' rule after each observed action: votes, claims, Seer divine results, and eliminations. Likelihood estimates are computed via Monte Carlo sampling over consistent role assignments.

$$P(\text{role} \mid \text{action}) \propto P(\text{action} \mid \text{role}) \cdot P(\text{role})$$

MCTS Agent (State-of-the-Art Baseline)

Uses Monte Carlo Tree Search with the UCT (Upper Confidence Trees) algorithm. For each decision, runs simulations of possible future game trajectories from the current state. Balances exploration of new branches against exploitation of promising ones using the UCB1 formula. Most published AIWolf competition winners use MCTS variants, making this our most competitive baseline.

$$\text{UCB1}(s, a) = Q(s, a) + c \cdot \sqrt{\ln N(s) / N(s, a)}$$

Logic-Based Agent (Symbolic Reasoning)

Uses propositional logic and forward chaining to derive conclusions from observed facts. Maintains a knowledge base of statements about claims, votes, and game events, then applies inference rules to deduce role assignments and identify lying players.

Contradiction handling: Uses tagged hypothetical reasoning. Claims are stored with their source — for example, `Claims(P3, Wolf(P5))` rather than `Wolf(P5)`. When two players make incompatible claims, the agent does not need to decide which is true; it derives that at least one of them must be deceptive, narrowing the suspect pool. This avoids the exponential complexity of full Truth Maintenance Systems while preserving useful inference.

Sample inference rules:

- If `Claims(P1, Seer)` AND `Claims(P2, Seer)` AND $P1 \neq P2 \rightarrow$ at least one of $\{P1, P2\}$ is on the wolf team

- If P claims Seer divined Q as wolf, AND Q is later revealed as wolf → P's claim is verified
- If P's claim conflicts with verified facts → P is on the wolf team
- If only one Seer claimant remains uncontradicted by late game → trust their divine results

4. Adversarial Wolf Strategies

For Experiment 4 (Targeted Deception Stress Test), we implement specialized wolf-only strategies that exhibit specific deceptive behaviors. These let us verify the suspicion module catches the patterns it was designed to detect.

- **Bus-Driver Wolf:** Votes against fellow wolves 20% of the time to deflect suspicion. Designed to test the Protection Patterns detector (D4).
- **False-Claimer Wolf:** Claims Seer role in 50% of games to mislead villagers with fabricated divine results. Tests the Claim Consistency detector (D5).
- **Coordinated Wolves:** Two wolves subtly defend each other through voting and discussion patterns. Tests D4 in a more realistic multi-agent setting.

5. The Suspicion Module

The suspicion module is a standalone class that maintains a scalar suspicion score $\sigma(\mathbf{p}) \in [0, 1]$ for each player. A score of 0 means "fully trusted," 1 means "highly suspicious of strategic deception." All players start at $\sigma = 0.5$ (neutral prior).

Key insight: Suspicion is not the same as "I think they're a werewolf." A player can have high role-belief (statistically likely wolf) but low behavioral suspicion (acting honestly), or vice versa. The agent uses both signals together.

A pure Bayesian agent only asks "what's the probability this player is a wolf based on their actions?" But Bayesian inference assumes a model of how each role acts. Wolves who play *exactly like villagers* are invisible to pure Bayesian. The suspicion module captures something different: deliberate strategic patterns that emerge over time, regardless of role.

6. The Five Deception Detectors

Each round, the suspicion module computes five behavioral signals from observed actions. Each detector outputs a value indicating evidence of deception.

D1 — Vote-Accusation Mismatch

Tracks whether a player's stated accusations match their actual votes. Honest players vote consistent with their stated beliefs. Wolves often accuse one player publicly to seem engaged, then vote for a different player to protect their teammate.

$$D1(p) = \text{mismatches}(p) / \text{rounds}(p)$$

D2 — Bandwagon Index

Measures whether a player makes independent voting decisions or always follows the existing majority. When a clear majority forms before a player votes, do they join it? Wolves in hiding often bandwagon to avoid drawing attention.

$$D2(p) = \text{times_followed_majority}(p) / \text{applicable_rounds}(p)$$

D3 — Pressure-Triggered Accusations

Tracks correlation between (1) how much suspicion is on player p and (2) how aggressively p accuses others. Positive correlation suggests deflection — only attacking others when cornered.

$$D3(p) = \text{correlation}(\text{suspicion_on_p}, \text{accusations_by_p})$$

D4 — Protection Patterns

For each pair (p, q), tracks whether p consistently shields q. Werewolves know each other's identities and may subtly protect teammates. Persistent protection between a specific pair suggests hidden alliance.

$$D4(p) = \max \text{ over } q \text{ of } (\text{times_p_defended_q} / \text{rounds})$$

D5 — Claim Consistency

When a player claims a role, tracks whether subsequent game events contradict their claim. A false Seer claim eventually surfaces when their stated divine results don't match revealed roles.

$$D5(p) = \text{contradicted_claims}(p) / \text{total_claims}(p)$$

7. Suspicion Update Mechanics

Composite Evidence

After each round, the agent computes a composite evidence score for each player by weighting the five detector outputs:

$$E(p) = a1 \cdot D1(p) + a2 \cdot D2(p) + a3 \cdot \max(D2(p) - 0) + a4 \cdot D4(p) + a5 \cdot D5(p)$$

The detector weights **a1** through **a5** are determined empirically via grid search, not chosen a priori. The optimal combination is found through Experiment 5 (Hyperparameter Sensitivity).

Asymmetric Update Rule

Suspicion updates with deliberate asymmetry — it rises faster than it falls. This encodes the insight that a single deceptive action provides stronger evidence than a single honest one, because honest behavior is the default expectation.

$$\begin{aligned} \text{If } E(p) > 0.5: & \quad \sigma(p) \leftarrow \min(1.0, \sigma(p) + 0.15 \cdot E(p)) \\ \text{If } E(p) \leq 0.5: & \quad \sigma(p) \leftarrow \max(0.0, \sigma(p) - 0.05 \cdot (1 - E(p))) \end{aligned}$$

Combined Decision Score

The suspicion-enhanced agent combines its base belief and suspicion into a single decision score:

$$\text{score}(p) = w1 \cdot \text{base_belief}(p) + w2 \cdot \sigma(p)$$

For Bayesian agents, $\text{base_belief}(p) = P(p = \text{wolf})$. For Heuristic agents, it is the normalized accusation count. For MCTS, it is the normalized win-rate estimate. The optimal balance ($w1$, $w2$) is determined via grid search in Experiment 5.

How Suspicion Evolves Over a Game

Early game (Days 1-2): Suspicion stays near 0.5 for everyone — not enough behavioral data. Decisions are dominated by base belief.

Mid game (Days 2-3): Patterns emerge. Players who consistently mismatch votes, bandwagon, or get caught in claim contradictions accumulate suspicion. Honest players slowly drift toward $\sigma = 0$.

Late game (Days 4+): Suspicion becomes a strong signal. Deceptive players have a noticeable suspicion gap from honest players.

8. Experimental Plan

Six experiments validate the suspicion module from different angles. Each experiment answers a specific research question.

Experiment 1: Paired Agent Comparison (Headline Result)

Goal: Prove the suspicion module improves performance across multiple compatible base agents (Heuristic, Bayesian, MCTS).

Setup: For each compatible base agent, run two configurations: (A) all 8 players are the base agent; (B) all 8 players are the base agent with suspicion enabled. 10,000 games per configuration.

Metrics: Village/wolf win rates, statistical significance via chi-squared, 95% confidence intervals.

Experiment 2: Detector Ablation Study

Goal: Identify which detectors actually contribute to performance.

Setup: Take strongest base agent (likely Bayesian + Suspicion). Run six configurations: full module and five "leave-one-out" variants. 5,000 games per configuration.

Metrics: Win rate drop per detector, suspicion calibration accuracy, detector activation frequency.

Experiment 3: Mixed-Population Tournament

Goal: Test how the suspicion-enhanced agent performs in heterogeneous games against all five base agent types.

Setup: Each of 8 player slots randomly filled with different agent types. Logic-Based, Heuristic, Bayesian, MCTS, and Random all represented. Rotate positions across games. 10,000 games total.

Metrics: Final ranking by win rate, pairwise win rates, comparative survival.

Experiment 4: Targeted Deception Stress Test

Goal: Prove that the suspicion module specifically catches deceptive wolf strategies — not just generic improvements.

Setup: Force wolves to use specific deception strategies (Bus-Driver, False-Claimer). Compare base agents against suspicion-enhanced variants. 5,000 games per matchup.

Metrics: Wolf identification accuracy, specific detector activations, time-to-identification (which day the wolf was first correctly suspected).

Experiment 5: Hyperparameter Grid Search

Goal: Empirically determine optimal weights for both detector weights (a_1 - a_5) and decision weights (w_1 , w_2).

Setup: Two-stage grid search. Stage 1: vary detector weights from 0.0 to 0.5 in increments of 0.1. Stage 2: using best detector weights, vary decision weights w_1 and w_2 . 2,000 games per configuration.

Metrics: Win rate as function of weight configuration, suspicion calibration, optimal weight combination.

Experiment 6: Scalability (Optional)

Goal: Demonstrate the approach scales beyond the primary 8-player setup.

Setup: Run smaller versions of Experiments 1 and 2 with 5-player and 15-player games. 2,000 games per configuration.

9. Compute Budget

Experiment	Games	Est. Runtime	Priority
Exp 1: Paired Comparison	60,000	~6 hours	Critical
Exp 2: Detector Ablation	30,000	~3 hours	Critical
Exp 3: Mixed Population	10,000	~1 hour	Important
Exp 4: Deception Stress Test	20,000	~2 hours	Critical
Exp 5: Grid Search	60,000	~6 hours	Critical
Exp 6: Scalability	10,000	~3 hours	Optional
Total	190,000	~21 hours	—

Total compute fits within two overnight runs. Experiments can be parallelized across team members' machines if needed.

Implementation Notes

- **Self-contained Python engine:** No Java server, no socket connections. All agents run in-process for speed and easy debugging.
- **Mirrors AIWolf interface:** Same method names (initialize, update, day_start, talk, vote, etc.) and GameInfo structure, ensuring compatibility with the official platform if needed later.
- **CSV logging:** Every game logs all relevant data (winner, rounds, agent decisions, suspicion scores, beliefs) to CSV for downstream analysis.
- **Configurable weights:** All detector and decision weights are configuration parameters, not hardcoded, to support the grid search experiment.
- **Logic-Based contradiction handling:** Uses tagged hypothetical reasoning to keep complexity polynomial in number of claims.

10. 6-Week Timeline

Week	Phase	Key Deliverables
1	Foundation	AIWolf platform setup, literature collection (20+ papers), survey paper outline, agent design
2	Survey Paper Pt 1	Survey sections 1-4 drafted, agent architecture finalized, suspicion model formalized
3	Survey Submission	Survey paper submitted (Week 11). Self-contained engine + 5 base agents implemented
4	Core Implementation	Suspicion module complete, integration with Heuristic/Bayesian/MCTS, initial 1,000-game
5	Full Experiments	All 6 experiments executed (~190,000 games). Statistical analysis. Charts and tables prep
6	Paper & Demo	Research paper submitted (Week 14, IEEE format, 8+ pages). Live demo prepared

Risk Mitigation

Risk	Mitigation
Suspicion module shows no improvement	Abandon results still publishable as negative findings. Different weight configurations may
Grid search too compute-heavy	Use coarse-grained search first (0.1 increments). Refine only around the peak.
MCTS implementation too slow	Limit simulation depth and rollout count. Use simpler determinized MCTS if needed.
Logic-Based agent crashes on configurations	Tagging hypothetical reasoning is bounded by $O(\text{claims}^2)$. Fall back to suspicion-tagged
Implementation bugs in suspicion logic	Strong unit tests. Compare against Bayesian baseline (which is well-understood).

11. Evaluation Metrics

Metric	Formula	Purpose
Win rate	wins / total_games	Primary outcome
Wolf ID precision	correct_wolf_votes / total_votes	Vote accuracy
Wolf ID recall	wolves_voted_against / total_wolves	Coverage
Survival rate	rounds_survived / total_rounds	Influence on game
False accusation rate	innocent_acc / total_acc	Errors as villager
Suspicion calibration	corr($\sigma(p)$, is_wolf(p))	Suspicion quality
Time to identification	first_day_correctly_suspected	Detection speed

Statistical Reporting

All comparative results report 95% confidence intervals computed via bootstrap resampling. Significance tests use chi-squared for win rate comparisons and two-sample t-tests for continuous metrics. The minimum effect size for claiming improvement is 2 percentage points in win rate, with $p < 0.05$.

Expected Headline Results

Based on theoretical reasoning (subject to empirical validation):

- **Bayesian + Suspicion** outperforms pure Bayesian by 5-10% in village win rate
- **MCTS + Suspicion** shows smaller but still significant improvement (3-6%)
- **Heuristic + Suspicion** shows smaller improvement, possibly insignificant
- **Logic-Based** outperforms Heuristic but underperforms Bayesian — establishes a clear paradigm hierarchy
- **D1 (Vote Mismatch)** and **D4 (Protection Patterns)** emerge as dominant detectors
- Suspicion module catches Bus-Driver wolves 1-2 days earlier than pure Bayesian
- Optimal decision weights cluster around $w_1 = 0.5-0.7$, $w_2 = 0.3-0.5$

This document serves as the working plan for the MLR 555 research project. All technical specifications here are intended to guide implementation in the self-contained Python engine and provide context for AI-assisted development tools such as Claude Code.